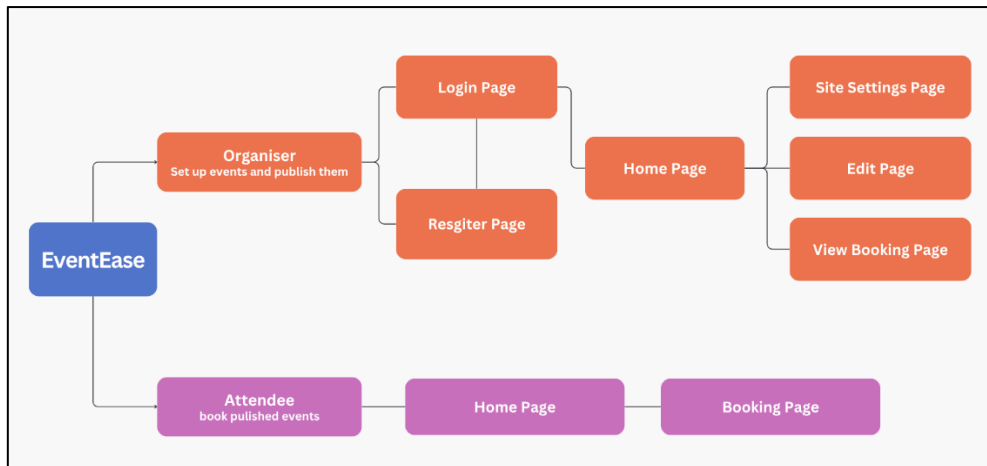


DATABASE MID TERM REPORT

1 Introduction

EventEase is a full-stack web application for booking and organising events. Events are booked by attendees and managed by organisers. Using a three-tier structure, it offers real-time ticket updates, booking history, and a secure login.

2 Summary of Pages

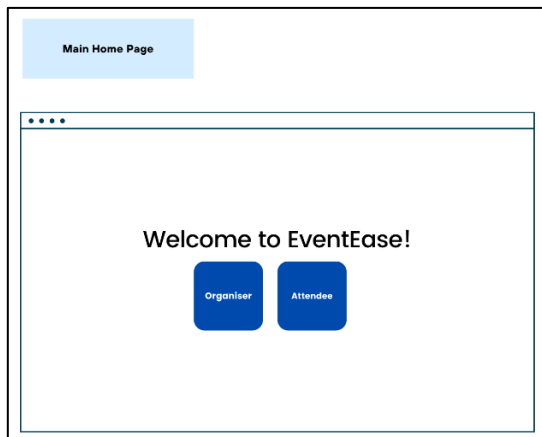


Img 1: Pages Summary

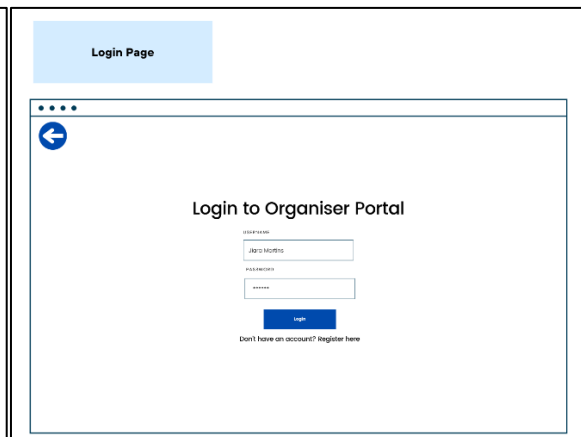
EventEase has two main user roles, Organiser and Attendee. Organisers manage events, settings, and bookings. Attendees browse and book events.

3 Wireframes

Wireframes show organiser and attendee pages sample layouts.



Img 2: Main home page



Img 3: login page

Register Page

Organiser Registration

USERNAME

John Martin

EMAIL

hello@freshgreentable.com

PASSWORD

Register

Already have an account? login here

Img 4: register page

Organiser Home Page

Welcome Name!

Site Name

Description

Logout

Settings

New Event

View Bookings

Published Events

Picture

Event date:

Created at:

Published at:

updated at:

Full : Concession:

Edit

Delete

Draft Events

Picture

Event date:

Created at:

Published at:

updated at:

Full : Concession:

Edit

Publish

Delete

Organiser Edit Event Page

Edit event

Image

Event Name

Event Description

Full Ticket

Quantity

Price

Concession Ticket

Quantity

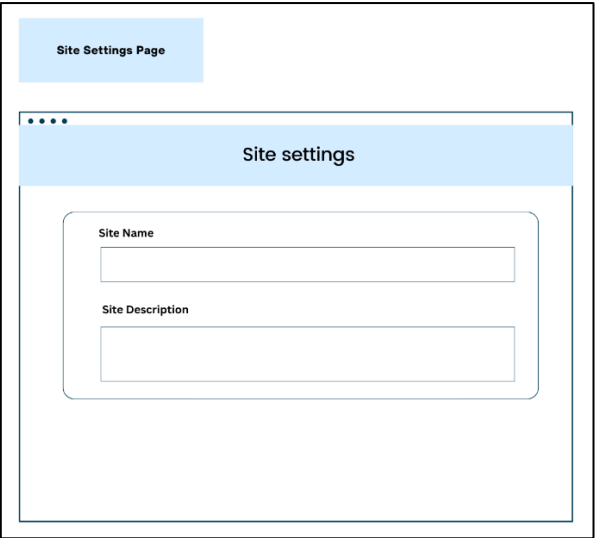
Price

Save as draft

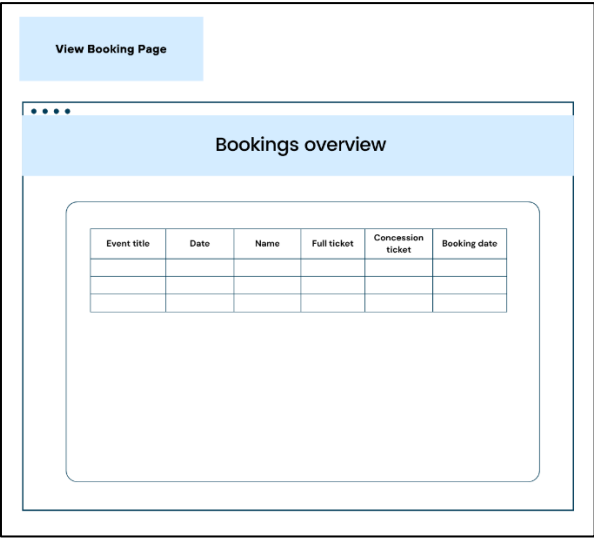
Publish

Img 5: organiser home page

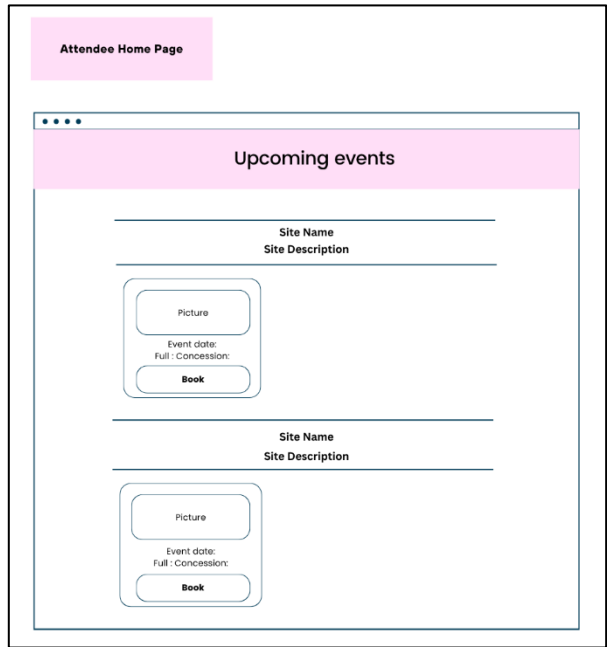
Img 6: edit event page



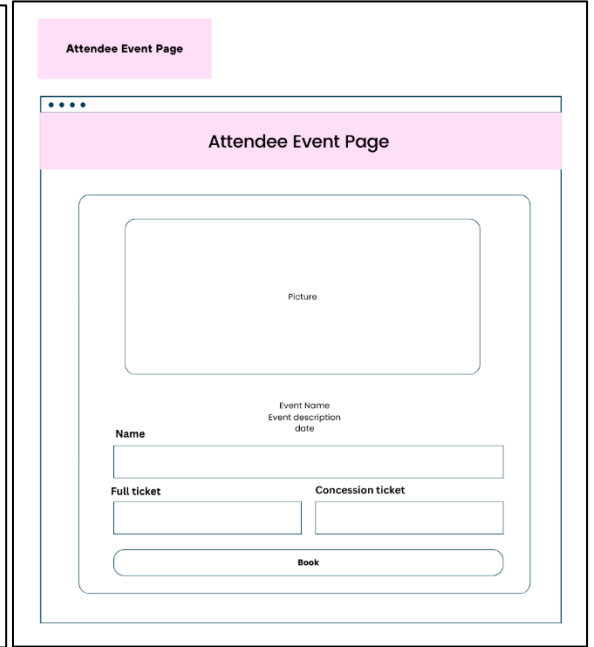
Img 7: site settings page



Img 8: view bookings page



Img 9: attendee home page

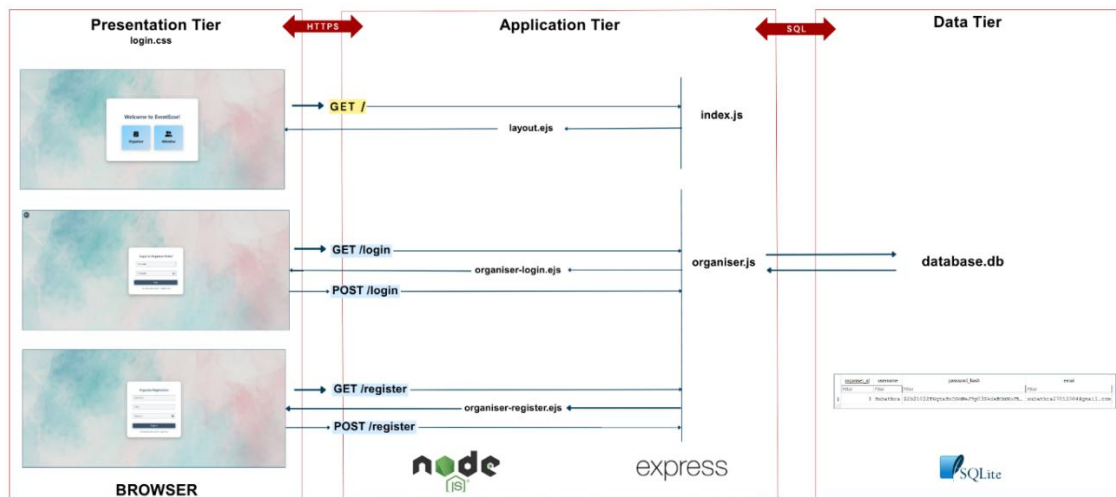


Img 10: event page

4 3-tier diagrams

This section shows the project's three-tier architecture, with the Presentation Tier, Application Tier, and Data Tier.

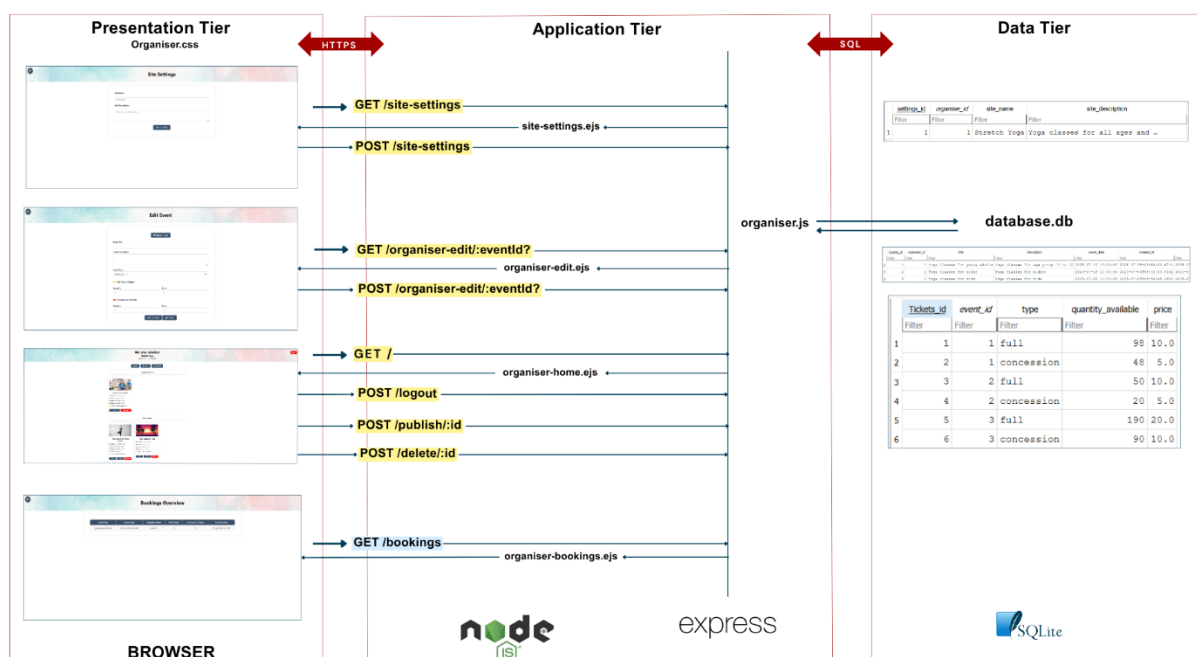
4.1 Login Pages



Img 11: diagram for login pages

The Presentation includes pages for registration, login, and homepage. GET routes provide EJS views, and Organiser.js uses database.db to manage POST /login and /register. Blue routes are the extension that indicate the security of the password.

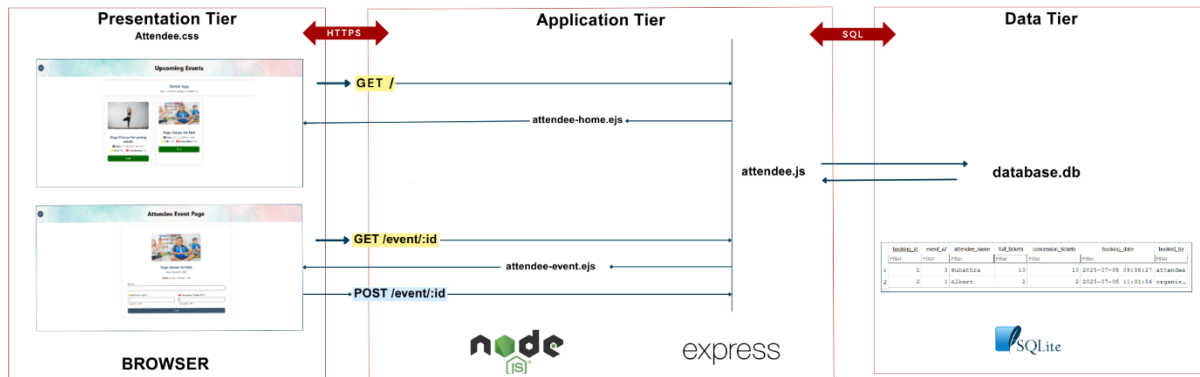
4.2 Organiser Pages



Img 12: diagram for organiser pages

EJS templates are used by Presentation Tier for organiser views. Settings, edits, bookings, logouts, publishing, and deletion are managed by routes. While the Application Tier makes use of Node.js and Express, Organiser.js, the Data Tier's database.db executes SQL queries. Extension Bookings are highlighted in blue.

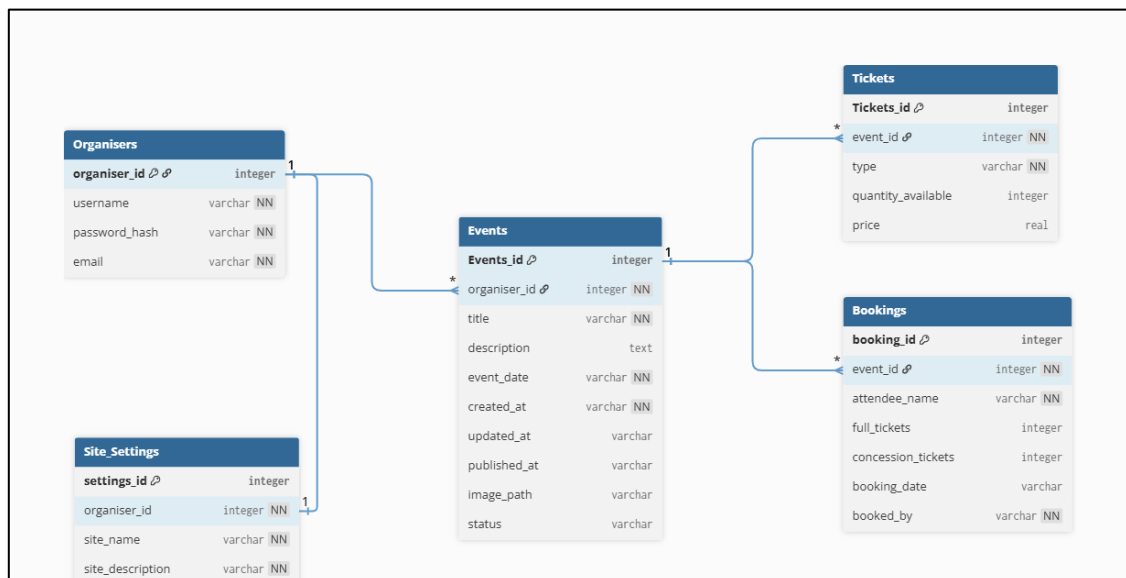
4.3 Attendee Pages



Img 13: diagram for attendee pages

Attendee-home and attendee-event presentation tiers shows events and booking information. Database.db data is retrieved by Attendee.js via GET / and /event/:id. Real-time ticket availability highlighted in blue is the extension.

5 ER Diagram



Img 14: ERD Diagram

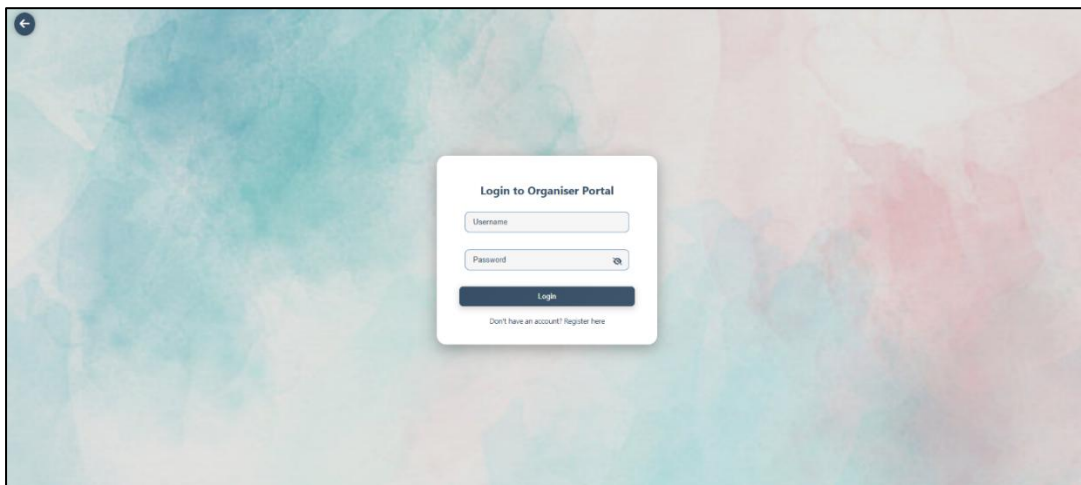
Organisers link one-to-many with Events and one-to-one with Site_Settings, storing hashed login credentials. Tickets link to events and track availability. Bookings store ticket counts and attendee information. Bookings and tickets have one to many relationships with events.

6 Extensions

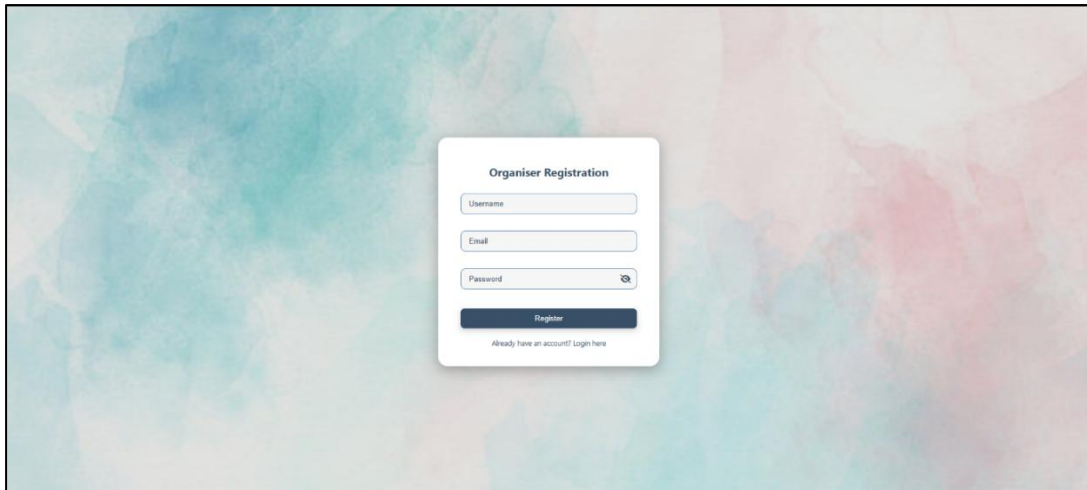
6.1 Extension 1: Login and Authentication

6.1.1 Description

To secure organiser-specific pages and routes, this extension uses express-session, bcrypt, and custom middleware to implement login and authentication. The /register and /login routes are used for account creation and login, respectively. Upon logging in, an organiser's ID is stored within the session. Bcrypt is a secure hashing algorithm for passwords. By limiting access to only authenticated users, the check_login middleware in organiser.js protects route access. Injection attacks and unauthorised access are prevented by input sanitisation and session validation. Instant user feedback for unsuccessful login attempts or registration problems is provided by messages. One significant feature that enhances data integrity is the check for duplicate usernames during registration. Session-based authentication, route protection, password hashing, and informative error messages that increase system dependability include the highlighted features.



Img 15: Login Page



Img 16: Register page

6.1.2 Code Snippets

File: `organiser.js` Code Line: 47 - 55

```
const check_login = (req, res, next) => {  
  // If the organiser id does not exists  
  if (!req.session.organiserid) {  
    // Return to login page  
    return res.redirect('/organiser/login');  
  }  
  // Continue to the next middleware if logged in  
  next();  
};
```

The `check_login` middleware looks for the session's organiser id. The user is taken to the login page if it cannot be found.

File: organiser.js Code Line: 73 - 115

```
router.post('/login', (req, res) => {  
  // Get the username and password from the submitted form  
  const { username, password } = req.body;  
  
  // Find the organiser by username in the database  
  db.get("SELECT * FROM Organisers WHERE username = ?", [username], (error, organiser) => {  
    // If error or organiser not found  
    if (error || !organiser) {  
      // Return to login page and show error message  
      return res.status(500).render('organiser-login', { message: 'Invalid credentials' });  
    }  
  
    // Compare the password with the hashed password  
    bcrypt.compare(password, organiser.password_hash, (error, resulting) => {  
      // If error or password is wrong  
      if (error || !resulting) {  
        // Return to the login page and show error message  
        return res.status(401).render('organiser-login', { message: 'Invalid credentials' });  
      }  
  
      // Set up session if username password is correct  
      req.session.organiserid = organiser.organiser_id;  
  
      // Check if site setting exist  
      db.get("SELECT * FROM Site_Settings WHERE organiser_id = ?", [organiser.organiser_id], (site_setting) => {  
        // If site setting not exist, insert default setting in db  
        if (!site_setting) {  
          db.run(  
            "INSERT INTO Site_Settings (organiser_id, site_name, site_description) VALUES (?, ?, ?)",  
            [organiser.organiser_id, 'My New Site', 'This is your description'],  
            () => {  
              // Redirect to organiser home page  
              res.redirect('/organiser/');  
            }  
          );  
        } else {  
          // If setting exist, redirect to organiser home page  
          res.redirect('/organiser/');  
        }  
      });  
    });  
  });  
});
```

Uses `bcrypt.compare()` to verify that the password entered matches the stored hash after retrieving the organiser by username from the database. If it does, the organiser id is saved to identify the user and a session is created.

```
router.post('/register', (req, res) => {
  // get username, password, email from the form
  const { username, password2, email } = req.body;

  // Check if username already exists
  db.get("SELECT * FROM Organisers WHERE username = ?", [username], (existing) => {
    // If username exists
    if (existing) {
      // Return the organiser register page with error message
      return res.status(400).render('organiser-register', { message: 'Username already exists' });
    }

    // If username does not exist, hash the password
    bcrypt.hash(password2, 10, (error, password_hashed) => {
      // If error for hash password
      if (error) {
        // Return organiser register page with error message
        return res.status(500).render('organiser-register', { message: 'Error creating password' });
      }

      // Insert new user record into the database
      db.run(
        "INSERT INTO Organisers (username, password_hash, email) VALUES (?, ?, ?)",
        [username, password_hashed, email],
        function (error) {
          // If error creating user
          if (error) {
            // Return organiser register page with error message
            return res.status(500).render('organiser-register', { message: 'Error creating user' });
          }

          // Send user to the login page again to log in
          res.redirect('/organiser/login');
        }
      );
    });
  });
});
```

This code verifies whether the username is already taken. If not, the password is hashed using salt factor 10 and bcrypt, and the organizer is saved before being redirected to the login page.

File: organiser.js Code Line: 531 - 557

```
router.post('/delete/:id', check_login, (req, res) => {  
  // Event ID from url  
  const event_Id = req.params.id;  
  
  // Delete all bookings for the event  
  db.run("DELETE FROM Bookings WHERE event_id = ?", [event_Id], function (error) {  
    // If error render organiser home page with error message  
    if (error) return res.status(500).render('organiser-home', { message: 'Fail to delete bookings' });  
  
    // Delete tickets related to the event  
    db.run("DELETE FROM Tickets WHERE event_id = ?", [event_Id], function (error) {  
      // If error, render organiser home page with error message  
      if (error) return res.status(500).render('organiser-home', { message: 'Fail to delete tickets' });  
  
      // SQL to delete the event  
      db.run("DELETE FROM Events WHERE Events_id = ?", [event_Id], function (error) {  
        // If error  
        if (error) {  
          // Render organiser home page with error message  
          return res.status(500).render('organiser-home', { message: 'Fail to delete event' });  
        }  
        // Redirect to organiser home page  
        res.redirect('/organiser/');  
      });  
    });  
  });  
});
```

By using check_login function as a parameter, this code protects organiser routes and protects data.

File: organiser-login.ejs Code Line: 18 - 37

```
<h1 class="page-title">Login to Organiser Portal</h1>  
  
<!-- POST the inputs from the form -->  
<form method="POST" action="/organiser/login">  
  <!-- Error message if the page has any errors-->  
  <% if (message) { %>  
    <div class="error-message"><%= message %></div>  
  <% } %>  
  
  <!-- Input for username -->  
  <input type="text" name="username" placeholder="Username" required class="input-field"><br><br>  
  <!-- Input for password with the toggle icon to view the password-->  
  <div class="password-wrapper">  
    <input type="password" name="password" id="password" placeholder="Password" required class="input-field">  
    <i class="fa-solid fa-eye-slash" id="eye_icon"></i>  
  </div>  
  
  <br>  
  <!-- Login button to login to account-->  
  <button type="submit" class="login-btn">Login</button>
```

Organiser's credentials are gathered by this form and sent to /organiser/login via POST. A message appears upon failed login.

```

<h1 class="page-title">Organiser Registration</h1>

<!-- Error message if the page has any errors-->
<div class="error-message">
  <% if (typeof message !== 'undefined') { %>
    <p><%= message %></p>
  <% } %>
</div>

<!-- POST the inputs from the form -->
<form method="POST" action="/organiser/register">
  <!-- Input for username -->
  <input type="username" name="username" placeholder="Username" required><br><br>
  <!-- Input for email -->
  <input type="email" name="email" placeholder="Email" required><br><br>
  <!-- Input for password with the toggle icon to view the password-->
  <div class="password-wrapper2">
    <input type="password" name="password2" id="password2" placeholder="Password" required class="input-field2">
    <i class="fa-solid fa-eye-slash" id="eye_icon"></i>
  </div>

  <br>
  <!-- Register button to register the details-->
  <button type="submit" class="register-btn">Register</button>

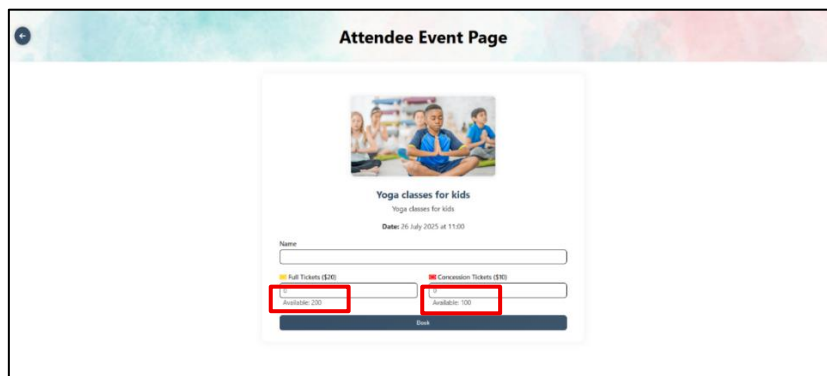
```

New organiser can register using this form by sending their username, email address, and password by POST to /organiser/register.

6.2 Extension 2: Show remaining tickets

6.2.1 Description

This extension makes it easier to manage and buy tickets by displaying to organisers and attendees how many of each type are left. There is now a `quantity_available` field in the Tickets table. When viewing an event, attendees can see the available tickets, and when booking, the system compares requested quantities with available tickets. Real-time data are maintained by reducing the number of tickets with valid bookings. Highlighted aspect would be the `quantity_available` field prevents competition during bookings by updating precisely to reflect real-time data. This extension gives organisers more control and encourages attendees to make timely purchases.



Img 23: Attendee event page

6.2.2 Code Snippets

File: attendee.js

Code Line: 98 - 165

```
// router.post('/event/:id', (req, res) => {
// Get id from url
const event_id = req.params.id;

// Get value from POST
const { attendee_name, full_no_tickets, concession_no_tickets } = req.body;

// Get referrer from query
const referrer = req.query.referrer || 'attendee';

// Convert tickets to integers
const full_number_tickets = parseInt(full_no_tickets) || 0;
const concession_number_tickets = parseInt(concession_no_tickets) || 0;

// If db values are null return error message
db.get('
SELECT
  IFNULL((SELECT quantity_available FROM Tickets WHERE event_id = ? AND type = "full"), 0) AS full_available,
  IFNULL((SELECT quantity_available FROM Tickets WHERE event_id = ? AND type = "concession"), 0) AS concession_available
', [event_id, event_id], (error, available) => {
  // If error return error message
  if (error) return res.status(500).render('attendee-event', { message: 'Error checking availability', event: null, referrer: referrer, success: false });

  // If quantity is more than the available ones
  if (full_number_tickets > available.full_available || concession_number_tickets > available.concession_available) {
    // Query event details again for error message
    const SQL_query = `
      SELECT Events.*,
        (SELECT quantity_available FROM Tickets WHERE event_id = Events.Events_id AND type = "full") AS full_tickets,
        (SELECT price FROM Tickets WHERE event_id = Events.Events_id AND type = "full") AS full_price,
        (SELECT quantity_available FROM Tickets WHERE event_id = Events.Events_id AND type = "concession") AS concession_tickets,
        (SELECT price FROM Tickets WHERE event_id = Events.Events_id AND type = "concession") AS concession_price
      FROM Events
      JOIN Site_Settings ON Events.organiser_id = Site_Settings.organiser_id
      WHERE Events.Events_id = ? AND Events.status = "published"
    `;
    db.get(SQL_query, [event_id], (error2, event) => {
      // If error return error message
      if (error2 || !event) {
        return res.status(404).render('attendee-event', { message: 'Event not found or not published', success: false });
      }
      // If not enough tickets, return error
      return res.render('attendee-event', { event, error: 'Not enough tickets available', referrer, success: false });
    });
    return;
  }

  // Insert the bookings in the sql
  db.run(
    `INSERT INTO Bookings (event_id, attendee_name, full_tickets, concession_tickets, booked_by)
    VALUES (?, ?, ?, ?, ?)
    `, [event_id, attendee_name, full_number_tickets, concession_number_tickets, referrer], function (err) {
      // If error return error message
      if (err) {
        return res.status(500).render('attendee-event', { message: 'Booking failed' });
      }

      // Update full tickets
      db.run(`UPDATE Tickets SET quantity_available = quantity_available - ? WHERE event_id = ? AND type = "full", [full_number_tickets, event_id]);
      // Update concession tickets
      db.run(`UPDATE Tickets SET quantity_available = quantity_available - ? WHERE event_id = ? AND type = "concession", [concession_number_tickets, event_id]);

      res.redirect('/attendee/event/${event_id}?success=true&referrer=${referrer}');
    });
  });
});
```

The booking process verifies that the requested tickets do not exceed availability, and inserts a booking if valid, and lowers quantity_available.

6.3 Extension 3: View Bookings

6.3.1 Description

Organisers can view all attendees' ticket bookings on a new page with this extension, which summarises the events, quantities, and tickets purchased. Using a SQL JOIN, Organiser.js implements routes to retrieve reservations for the events of the logged-in organiser. The organiser-bookings.ejs template helps organisers keep an eye on sales in real time by showing the data in a table. Highlighted aspect would be the SQL query ensures that only bookings for the events of the organiser are displayed. The table layout, provides a clear summary of attendees' activities, improves control.

Event Title	Event Date	Attendee Name	Full Tickets	Concession Tickets	Booking Date
Yoga classes for kids	26 July 2025 at 11:00	Subathra	10	10	03 July 2025 at 17:58

Img 25: Bookings overview page

6.3.2 Code Snippets

File: organiser.js Code Line: 565-589

```
router.get('/bookings', check_login, (req, res) => {
  // organiser id from session
  const organiser_id = req.session.organiserid;

  // SQL to get all the bookings and events details
  const SQL = `
    SELECT Bookings.attendee_name, Bookings.full_tickets, Bookings.concession_tickets, Bookings.booking_date, Events.title AS event_title, Events.event_date
    FROM Bookings
    JOIN Events ON Bookings.event_id = Events.event_id
    WHERE Events.organiser_id = ?
    ORDER BY Bookings.booking_date DESC
  `;

  // SQL to get the data from db
  db.all(SQL, [organiser_id], (error, bookings) => {
    // If error
    if (error) {
      // Render organiser booking page with error message
      return res.status(500).render('organiser-bookings', { message: 'fail to load bookings' });
    }

    // Render organiser booking page with all the bookings
    res.render('organiser-bookings', { bookings });
  });
});
```

Results are shown on the organiser's bookings page after this route combines Bookings and Events and filters by organiser ID.

File: organiser-bookings.ejs Code Line: 40-80

```
<table class="event-table">
  <!-- Headings for the table -->
  <thead>
    <tr>
      <th>Event Title</th>
      <th>Event Date</th>
      <th>Attendee Name</th>
      <th>Full Tickets</th>
      <th>Concession Tickets</th>
      <th>Booking Date</th>
    </tr>
  </thead>

  <!-- Body for the table -->
  <tbody>
    <!-- For each booking -->
    <% bookings.forEach(booking => { %>
      <!-- Show the event title -->
      <td><%= booking.event_title %></td>
      <!-- For formatted date and time for the event -->
      <%
        let date_book = new Date(booking.event_date.replace(' ', 'T'));
        let datePart = Date_book.toLocaleDateString('en-GB', { day: '2-digit', month: 'long', year: 'numeric' });
        let timePart = Date_book.toLocaleTimeString('en-GB', { hour: '2-digit', minute: '2-digit', hour12: false });
        <%= `${datePart} at ${timePart}` %>
      </td>
      <!-- Attendee name -->
      <td><%= booking.attendee_name %></td>
      <!-- Number of booked full tickets -->
      <td><%= booking.full_tickets %></td>
      <!-- Number of booked concession tickets -->
      <td><%= booking.concession_tickets %></td>
      <!-- Booking date -->
      <td><%= new Date(new Date(booking.booking_date).getTime() + 8 * 60 * 60 * 1000).toLocaleString('en-GB', { day: '2-digit', month: 'long', year: 'numeric', hour: '2-digit', minute: '2-digit' }) %></td>
    <% } %>
  </tbody>
</table>
```

This code shows all reservations in a table with event titles, attendee names, and ticket counts, allowing organisers to keep track of reservations.

7 Conclusion

The system's overall performance, scalability, and data access control are enhanced by JOIN queries and structured middleware. To provide a dependable event system with booking visibility, ticket tracking, and a secure organiser login, EventEase integrates frontend, server, and database.

Word Count: 998 Words

8 References

Code References:

Bamidele, O., 2023. Beginner Topic: File Upload With Multer In Nodejs. [online]. Available at: <https://dev.to/ngfizzy/beginner-topic-file-upload-with-multer-in-nodejs-99m> [Accessed 6 June 2025]

Malhotra, A., 2023. Using Multer to Store Files in Express: A Comprehensive Guide. [online]. Available at: <https://medium.com/@aman003malhotra/using-multer-to-store-files-in-express-a-comprehensive-guide-d1acd25ef4d5> [Accessed 6 June 2025]

Report References:

GeeksforGeeks, 2025. Introduction of 3-Tier Architecture in DBMS. [online] Available at: <https://www.geeksforgeeks.org/dbms/introduction-of-3-tier-architecture-in-dbms-set-2/> [Accessed 28 June 2025]